Advanced Database Systems:

Part III.3b Document Stores

Reinhard Pichler

Institute of Logic and Computation
"Databases & Artificial Intelligence" Group

Summer Semester 2026

TU WIEN Informatics

References

- Textbooks:
  - Pramod J. Sadalage and Martin Fowler:
    NoSQL Distilled
    Addison-Wesley 2013
  - Lena Wiese:
    Advanced Data Management for SQL, NoSQL, Cloud and Distributed Databases, de Gruyter, 2015
- Further resource:
  MongoDB documentation: https://docs.mongodb.com/
- Copyright of all figures is with one of these resources.

Contents

- **Basic Principles**
- Data Modelling in Document Stores
- MongoDB: Overview
- MongoDB: Basic Commands

Document Stores

- Document stores:
  - Similar to key-value stores but DBMS can understand the format of the value
  - Typical data formats: XML, JSON (JavaScript Object Notation), BSON (binary JSON)
  - Allows advanced querying (e.g.: filters on field values) and relationships (references) between data objects
  - Joins usually not supported for performance reasons; consequence: denormalizing data when modelling or combining documents on application level.
  - Atomicity usually (only) of documents guaranteed
- In this lecture: look at MongoDB

Typical Use Cases

- Log data:
  - K/V-store: storing log information as pure text
  - Document store: extract different fields from a log file, e.g.: user, time, path, action, etc.
- e-commerce, e.g.:
  - product catalog, shopping cart
- Content management:
  - storing all kinds of content and metadata of web sites
  - "Store and serve any type of content, build any feature, serve it any way you like, from a single database."

see also https://www.mongodb.com/use-cases

JSON (JavaScript Object Notation),

- Sytnax (cf. www.json.org):
  - JSON object = collection of name/value pairs in { }
  - the name is a string
  - the value can be
    - of a simple type: a number, string, true/false
    - an array: = sequence of values in [ ]
    - a nested JSON object: in { }
- Remark:
  - Two fields with the same name in principle allowed (in particular, also in BSON used by MongoDB)
  - most MongoDB interfaces disallow duplicate names because of the used representation (e.g., hash table)

Structure of the Database: SQL vs. MongoDB

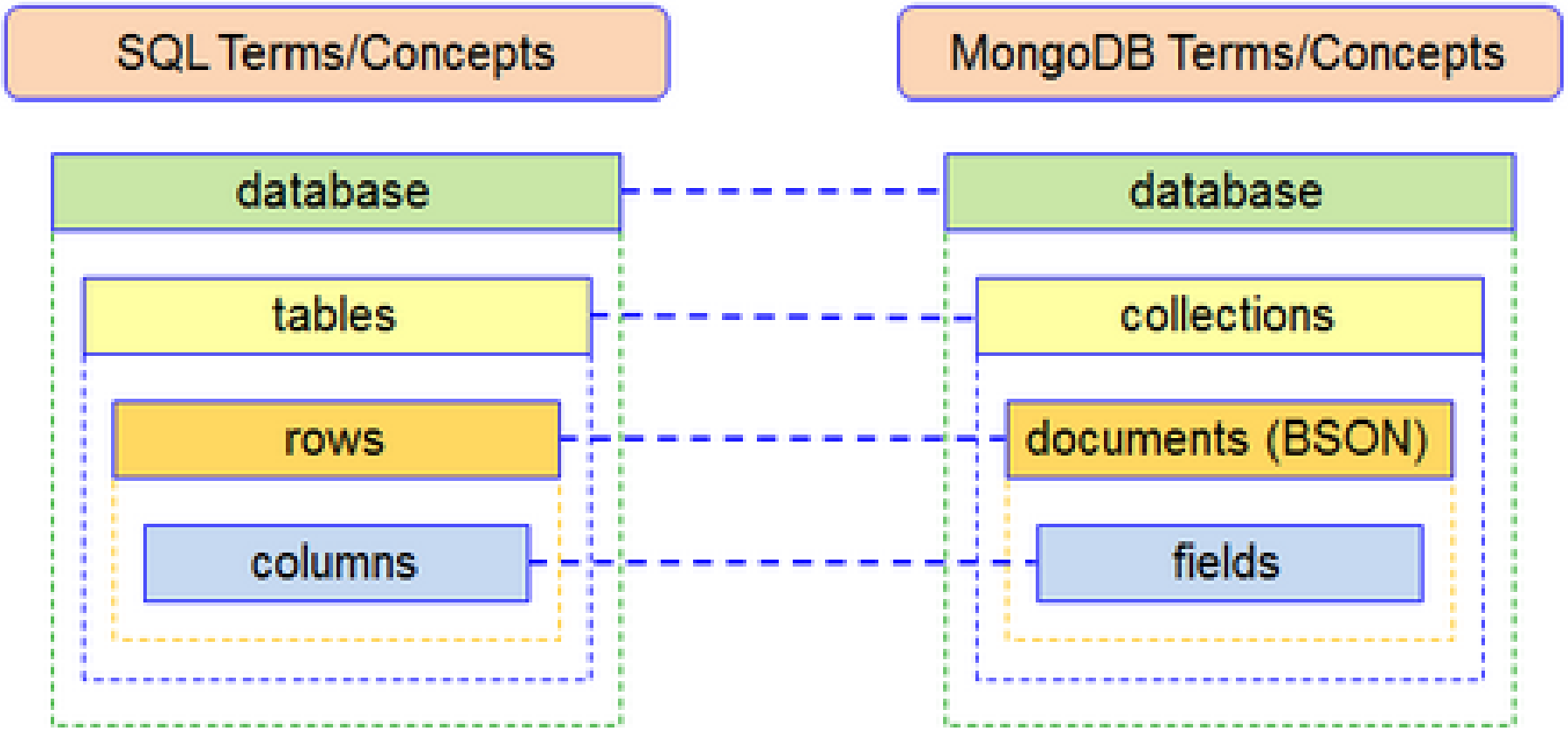
Relational Database

Student_Id	Student_Name	Age	College
1001	Chaitanya	30	Beginnersbook
1002	Steve	29	Beginnersbook
1003	Negan	28	Beginnersbook

MongoDB

```
{
 "_id": ObjectId("....."),
 "Student_Id": 1001,
 "Student_Name": "Chaitanya",
 "Age": 30,
 "College": "Beginnersbook"
}
{
 "_id": ObjectId("....."),
 "Student_Id": 1002,
 "Student_Name": "Steve",
 "Age": 29,
 "College": "Beginnersbook"
}
{
 "_id": ObjectId("....."),
 "Student_Id": 1003,
 "Student_Name": "Negan",
 "Age": 28,
 "College": "Beginnersbook"
}
```

Structure of the Database: SQL vs. MongoDB (cont.)



Contents

- Basic Principles
- **Data Modelling in Document Stores**
- MongoDB: Overview
- MongoDB: Basic Commands

Data Modelling

- Basic principles:
  - Keep intended data access in mind
  - aim for atomic reads/writes if possible
  - various ways to model 1:n and n:m relationships (choose the most suitable one for expected workload)
  - denormalization is an option
  - respect the max. document size (BSON): 16 MB
  - use document validation, e.g.: warnings/errors if a document gets an unexpected field: check if this is a bug or a feature of the schema design
  - decide which indexes shall be created

Modelling 1:n Relationships

- Relational model:
  - typically 2 tables (one for each of the entities)
  - model 1:n relationship with a foreign key
- MongoDB: several possibilities:
  - two collections with reference (analogous to a foreign key); reference can be _id or some key attribute: recommended for "one-to-millions"
  - array of references in the "one-side" document, e.g: array with _ids or order-numbers in customer document; recommended for "one-to-thousands"
  - embed an array of documents in the other document, e.g.: array of order-documents in a customer document; for "one-to-few"

Modelling n:m Relationships

- Relational model:
  - typically three tables: auxiliary table for the relationship, one table for each entity,
  - foreign keys from auxiliary table into each of the others
- MongoDB: several possibilities, e.g.:
  - three collections with references as in relational case
  - references as for 1:n relationship, but from both sides, e.g.: store references to courses plus partial information on courses in student document (i.e., array of course IDs + course-info) and store references to students in course document (i.e., array of student IDs).

Modelling 1:n Relationships (cont.)

- Combination of two referencing methods:
  - useful if fast access from "both sides" is required

```
db.persons.insertOne()
{
    _id: ObjectID("AAF1"),
    name: "Kate Miller",
    tasks [
            ObjectID("ADF9"),
            ObjectID("AE02"),
            ObjectID("AE73")
            // etc
    ]
}
```

```
db.tasks.insertOne()
{
    _id: ObjectID("ADF9"),
    description: "Write lesson plan",
    due_date:  ISODate("2014-04-01"),
    owner: ObjectID("AAF1")
    // Reference to persons document
}
```

Denormalization

- Idea:
  - allow redundancy to avoid joins when querying
  - disadvantage: updates are not atomic anymore
  - makes most sense for high read-to-write ratio
- Example 1: denormalizing into "1-side"

```
db.products.insertOne()
{     name : 'left-handed smoke shifter',
      catalog_number: 1234,
      parts : [
         { id : ObjectID('AAAA'), name : '#4 grommet' },
         { id: ObjectID('F17C'), name : 'fan blade assembly' },
         { id: ObjectID('D2AA'), name : 'power switch' },
         // etc
      ] }
```

Denormalization (cont.)

- Example 2: denormalizing into "many-side"
  - increases the cost of updates yet further
  - high read-to-write ratio even more important

```
db.parts.insertOne()
{  _id : ObjectID('AAAA'),
   partno : '123-aff-456',
   name : '#4 grommet',
   products: [
      { p_name : 'left-handed smoke shifter',  p_catalog_number: 1234},
      { p_name:  … , p_datalog_number: … } ] ,
   qty: 94,
   cost: 0.94,
   price: 3.99
}
```

Contents

- Basic Principles
- Data Modelling in Document Stores
- **MongoDB: Overview**
- MongoDB: Basic Commands

Introduction to MongoDB

- Name: from "humongous ": = enormous, gigantic.
- Data format: stores data in BSON
- Brief history:

  2009: version 1 of MongoDB released,

  constantly extended feature set, e.g.:
  - Sharding, replica sets (2010)
  - Document validation (2015)
  - MongoDB Atlas (MongoDB as a cloud service, 2016)
  - Multi-document ACID transactions (2018)
  - Distributed ACID transactions (2019)

Document Validation

- MongoDB allows to specify rules for validating documents:
  - defined for a collection
  - in the form of a JSON Schema
  - allows specification of action: error/warning

```
db.createCollection( "contacts", {
    validator: { $jsonSchema: {
      bsonType: "object",
      required: [ "phone", "name" ],
      properties: {
        phone: {
          bsonType: "string",
          description: “string; required"
        },
        name: {
          bsonType: "string",
          description: "string; required"
        }
      }
    } },
    validationAction: "warn"
} )
```

Replication

MongoDB uses Primary-Secondary distribution:

- MongoDB allows the definition of replica sets, i.e.: one primary server and several secondary servers.
- Writes are only executed at the primary and then propagated to the secondaries by operation logs (i.e, execute the primary's operations in the same order)
- When the secondary servers do not hear from the primary for more than 10 seconds (configurable), they hold an election to vote for a new primary.
- A replica set may also contain an arbiter, i.e.: a node that has a vote in an election but carries no data.

Read Preferences

- MongoDB allows definition of Read preferences to define the read behaviour of a replica set:
  - primary (= default) resp. secondary: all reads are executed on the primary resp. secondary server;
  - primaryPreferred resp. secondaryPreferred: reads are executed on primary resp. secondary *if available*;
  - nearest: reads are executed on server which is nearest in terms of network latency.
  - Caveat: secondary may return stale data
- Examples: db.getMongo().setReadPref('secondaryPreferred')

  db.my_collection.find({ … }).readPref("secondary")

  (read preference for entire session or single command)

Write Concerns

- Write concerns:
  - A write operation is acknowledged in a replica set once the write is acknowledged by the majority of servers.
  - This behaviour can be changed by write concerns on different levels (single operation, transaction, session).
  - Values, in particular "majority" or number of servers ("majority" = $\lfloor N/2 \rfloor + 1$ with N = #servers in replica set)
- Example:

```
db.books.insert(
    { name: "Mastering MongoDB", isbn: "1001" },
    { writeConcern: {w: 2, j: true, wtimeout: 5000} } )
```

write is acknowledged when 2 servers have written the insert to their on-disk journal. If the timeout of 5000 ms is exceeded, an error is returned.

Read Concerns

- Read concerns:
  - Can be defined for read operations (queries), e.g.: find(), aggregate(), count(), etc.;
  - Values, in particular:
    "local" (= default): current value on the read target,
    "majority" (most recent value on a majority of the servers).
  - default: "local".
- Example: `db.persons.find( { age: 22}).readConcern("majority").maxTimeMS(5000)`

  read the most recent values from a majority of servers; returns an error, if the timeout of 5000 ms is exceeded.

Transactions

- Multi-document transactions with ACID properties have been introduced in version 4.0 (2018).
- Atomicity of single document access is guaranteed also without transactions (and recommended for performance).
- Example:

```
session = db.getMongo().startSession( )
session.startTransaction( { writeConcern: { w: "majority" } } )
  db.books.insertOne( [{ title: "Alice in Wonderland" , isbn: 9780})
  db.customers.insertOne ( { name: “bob" , age: 22, location: “vienna"})
session.commitTransaction()  // respectively: session.abortTransaction()
session.endSession()
```

Sharding

- MongoDB supports sharding (= horizontal scaling): shard = subset of the data, deployed as a replica set
- A sharded cluster consists of the following elements:
  - Two or more shards
  - One or more query routers ("mongos")
  - A replica set of config servers (for metadata and configuration data of the entire cluster).
- MongoDB shards data on collection level; shards are defined via a shard key (= field that exists in every document in the collection to be sharded).

Contents

- Basic Principles
- Data Modelling in Document Stores
- MongoDB: Overview
- **MongoDB: Basic Commands**

Using MongoDB

- 3 editions:
  - MongoDB Community: open source;
    or free download for Windows, Linux, macOS
  - MongoDB Enterprise Advanced: full functionality
  - MongoDB Atlas: cloud edition
- Basic functionality is (largely) provided by MongoDB Community, e.g.: ACID transactions, indexes, schema validation, replica sets, sharding
- Missing features with MongoDB Community, e.g.:
  - commercial support, long-term support
  - advanced monitoring (performance)
  - advanced management tools (deployment, backup,…)

Basic MongoDB Commands (1)

To do this	Run this command	Example
Start server	mongod	mongod
Start shell	mongosh	mongosh
Show current database	db	db
Select database (and create if not exists)	use <database name>	use mydb
Execute a JavaScript file	load(<filename>)	load (myscript.js)
Create a collection	db.createCollection (<collection name>)	db.createCollection ("books")
show all databases	show dbs	show dbs
Show all collections in current database	show collections	show collections

Basic MongoDB Commands (2)

To do this	Run this command	Example
Insert new document in a collection	db.collection.insertOne(*<document>*)	db.books.insertOne({isbn: 9780, title: "Alice in Wonderland",..})
Insert multiple documents into a collection	db.collection.insertMany([*<document1>*, *<document2>*, ...]) -or- db.collection.insert([*<document1>*, *<document2>*, ...])	db.books.insertMany([{isbn: 9780, title: "Alice in Wonderland", ...}, {isbn: 9781, title: ...}])
Show all documents in the collection	db.collection.find ()	db.books.find()
Filter documents by field value condition	db.collection.find(*<query>*)	db.books.find({title: "Alice in Wonderland"})

Basic MongoDB Commands (3)

To do this	Run this command	Example
Show only some fields of matching documents	db.collection.find(*<query>, <projection>)*	db.books.find({title: "Alice in Wonderland"}, {title:true, _id:false})
Show first document that matches the query condition	db.collection.findOne(*<query>, <projection>*)	db.books.findOne({}, {_id:false})
Comparison operators	$lt, $lte, $gt, $gte, $eq, $ne	db.books.find ({price: {$gt: 10.0} }))
Combining conditions	$and, $or, $not	... .find({ $and: [{ price: {$gte: 10} }, { price: {$lte: 20} }]})
		... find({ price: { $not: { $gt: 1.99 } } })

Basic MongoDB Commands (4)

To do this	Run this command	Example
Add or update specific fields of a single document	db.collection.updateOne (*<query>, <update>*)	db.books.updateOne({title : Alice in W... "}, {$set : {category: "Fiction"}})
Remove certain fields of a single document	{$unset: {field:""}}	db.books.updateOne({title : "Alice in W..."}, {$unset : {author:""}})
Apply update to several documents	db.collection.updateMany(*<query>, <update>*)	db.books.updateMany({price: {$gt: 10.0}}, {$mul: {price: 0.9}})
Apply update to all documents	empty document as query	db.books.updateMany({}, {$inc: {price: 2.0}})

Basic MongoDB Commands (5)

To do this	Run this command	Example
Delete the first document matching a query condition	db.collection.deleteOne (*<query>*)	db.books.deleteOne ({title : "Alice in Wonderland"})
Delete all documents matching a query condition	db.collection.deleteMany (*<query>*)	db.books.deleteMany ({category: "Fiction"})
Delete all documents in a collection	empty document as query	db.books.deleteMany ({})
Remove a (not necessarily emtpy) collection	db.collection.drop()	db.books.drop()

Basic MongoDB Commands (6)

To do this	Run this command	Example
Create an index	db.collection.createIndex({indexField:type}) Type 1 means ascending; -1 means descending	db.books.createIndex({ title:1})
Show all indexes in a collection	db.collection.getIndexes()	db.books.getIndexes()
Drop an index	db.collection.dropIndex({indexField:type})	db.books.dropIndex({author: -1})
Drop all indexes except for _id	db.collection.dropIndexes ()	db.books.dropIndexes()
number of documents in the collection	cursor.count()	db.books.find().count()

Summary

- main characteristics; typical data formats, typical use cases
- SQL vs. MongoDB
- Data modelling:
  - basic principles
  - 1:n / n:m relationships
  - denormalization
- Basic MongoDB commands:
  - create a collection, insert a document
  - find, findOne: $lt, $gt, eq, $ne, $and, $or, $not, …
  - updateOne, updateMany, deleteOne, deleteMany
  - etc.